

Lecture (08)

chapter 11 : More About Classes and Object-Oriented Programming 2

Topics

- 11.5 Copy Constructors
- 11.6 Operator Overloading
- 11.7 Type Conversion Operators
- 11.8 Convert Constructors
- 11.9 Aggregation and Composition
- 11.10 Inheritance
- 11.11 Protected Members and Class Access
- 11.12 Constructors, Destructors, and Inheritance
- 11.13 Overriding Base Class Functions

11.5 Copy Constructors

- Special constructor used when a newly created object is initialized to the data of another object of same class
- Default copy constructor copies field-to-field, using memberwise assignment
- The default copy constructor works fine in most cases

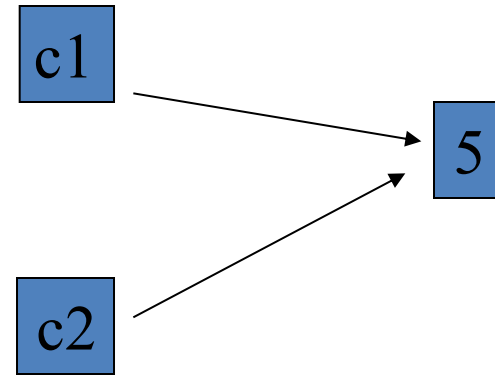
Copy Constructors

Problems occur when objects contain pointers to dynamic storage:

```
class CpClass
{
    private:
        int *p;
    public:
        CpClass(int v=0)
            { p = new int; *p = v; }
        ~CpClass() {delete p;}
};
```

Default Constructor Causes Sharing of Storage

```
CpClass c1(5);  
if (true)  
{  
    CpClass c2=c1;  
}  
// c1 is corrupted  
// when c2 goes  
// out of scope  
and  
// its destructor  
// executes
```



Problems of Sharing Dynamic Storage

- Destructor of one object deletes memory still in use by other objects
- Modification of memory by one object affects other objects sharing that memory

Programmer-Defined Copy Constructors

- A copy constructor is one that takes a reference parameter to another object of the same class
- The copy constructor uses the data in the object passed as parameter to initialize the object being created
- Reference parameter should be **const** to avoid potential for data corruption

Programmer-Defined Copy Constructors

- The copy constructor avoids problems caused by memory sharing
- Can allocate separate memory to hold new object's dynamic member data
- Can make new object's pointer point to this memory
- Copies the data, not the pointer, from the original object to the new object

Copy Constructor Example

```
class CpClass
{
    int *p;
public:
    CpClass(const CpClass &obj)
    { p = new int; *p = *obj.p; }
    CpClass(int v=0)
    { p = new int; *p = v; }
    ~CpClass() {delete p;}
};
```

Copy Constructor – When Is It Used?

A copy constructor is called when

- An object is initialized from an object of the same class
- An object is passed by value to a function
- An object is returned using a **return** statement from a function

11.6 Operator Overloading

- Operators such as `=`, `+`, and others can be redefined for use with objects of a class
- The name of the function for the overloaded operator is **`operator`** followed by the operator symbol, *e.g.*,
 `operator+` is the overloaded `+` operator and
 `operator=` is the overloaded `=` operator

Operator Overloading

- Operators can be overloaded as
 - instance member functions, or as
 - friend functions
- The overloaded operator must have the same number of parameters as the standard version. For example, **operator=** must have two parameters, since the standard = operator takes two parameters.

Overloading Operators as Instance Members

A binary operator that is overloaded as an instance member needs only one parameter, which represents the operand on the right:

```
class OpClass
{
    private:
        int x;
    public:
        OpClass operator+(OpClass right) ;
};
```

Overloading Operators as Instance Members

- The left operand of the overloaded binary operator is the calling object
- The implicit left parameter is accessed through the **this** pointer

```
OpClass OpClass::operator+(OpClass r)
{
    OpClass sum;
    sum.x = this->x + r.x;
    return sum;
}
```

Invoking an Overloaded Operator

- Operator can be invoked as a member function:

```
OpClass a, b, s;  
s = a.operator+(b);
```

- It can also be invoked in the more conventional manner:

```
OpClass a, b, s;  
s = a + b;
```

Overloading Assignment

- Overloading the assignment operator solves problems with object assignment when an object contains pointer to dynamic memory.
- Assignment operator is most naturally overloaded as an instance member function
- It needs to return a value of the assigned object to allow cascaded assignments such as

`a = b = c;`

Overloading Assignment

Assignment overloaded as a member function:

```
class CpClass
{
    int *p;
public:
    CpClass(int v=0)
    { p = new int; *p = v; }
    ~CpClass() { delete p; }
    CpClass operator=(CpClass) ;
};
```

Overloading Assignment

Implementation returns a value:

```
CpClass CpClass::operator=(CpClass r)
{
    *p = *r.p;
    return *this;
};
```

Invoking the assignment operator:

```
CpClass a, x(45);
a.operator=(x); // either of these
a = x;         // lines can be used
```

Notes on Overloaded Operators

- Overloading can change the entire meaning of an operator
- Most operators can be overloaded
- Cannot change the number of operands of the operator
- Cannot overload the following operators:

`?: . .* sizeof`

Overloading Types of Operators

- `++`, `--` operators overloaded differently for prefix vs. postfix notation
- Overloaded relational operators should return a **bool** value
- Overloaded stream operators `>>`, `<<` must return **istream**, **ostream** objects and take **istream**, **ostream** objects as parameters

Overloaded [] Operator

- Can be used to create classes that behave like arrays, providing bounds-checking on subscripts
- Overloaded [] returns a reference to object, not an object itself

11.7 Type Conversion Operators

- **Conversion Operators** are member functions that tell the compiler how to convert an object of the class type to a value of another type
- The conversion information provided by the conversion operators is automatically used by the compiler in assignments, initializations, and parameter passing

Syntax of Conversion Operators

- Conversion operator must be a member function of the class you are converting from
- The name of the operator is the name of the type you are converting to
- The operator does not specify a return type

Conversion Operator Example

- To convert from a class `IntVal` to an integer:

```
class IntVal
{
    int x;
public:
    IntVal(int a = 0) {x = a;}
    operator int() {return x;}
};
```

- Automatic conversion during assignment:

```
IntVal obj(15); int i;
i = obj;    cout << i; // prints 15
```

11.8 Convert Constructors

Convert constructors are constructors that take a single parameter of a type other than the class in which they are defined

```
class CCClass
{   int x;
    public:
        CCClass()                //default
        CCClass(int a, int b);
        CCClass(int a);          //convert
        CCClass(string s);       //convert
};
```

Example of a Convert Constructor

The C++ **string** class has a convert constructor that converts from C-strings:

```
class string
{
    public:
        string(char *);    //convert
        ...
};
```

Uses of Convert Constructors

- They are automatically invoked by the compiler to create an object from the value passed as parameter:

```
string s("hello");    //convert C-string  
CCClass obj(24);      //convert int
```

- The compiler allows convert constructors to be invoked with assignment-like notation:

```
string s = "hello";   //convert C-string  
CCClass obj = 24;     //convert int
```

Uses of Convert Constructors

- Convert constructors allow functions that take the class type as parameter to take parameters of other types:

```
void myFun(string s); // needs string
                        // object
myFun("hello");       // accepts C-string

void myFun(CCClass c);
myFun(34);             // accepts int
```

11.9 Aggregation and Composition

- **Class aggregation:** (Weak Relationship) An object of one class owns an object of another class
- Objects are loosely connected. If the main object is destroyed → the other still exists
- A **Student has a Laptop**,
- **Class composition:** (Strong Relationship) A form of aggregation where the enclosing class controls the lifetime of the objects of the enclosed class
- Supports the modeling of 'has-a' relationship between classes – enclosing class 'has a(n)' instance of the enclosed class
- One object **owns** another object → The second object **cannot exist alone**
- Objects are **strongly connected** → If the main object is destroyed → the other is destroyed
- A **House has Rooms**, **Car has Engin**

Feature	Aggregation	Composition
Relationship	Weak (uses)	Strong (owns)
Lifetime	Independent	Dependent
Creation	Outside class	Inside class
Memory	Usually pointer/reference	Direct object
Example	Student → Laptop Doctor → dept Student → course	Car → Engine Class → student Car → wheels, doors

Object Composition

```
class StudentInfo
{
    private:
        string firstName, LastName;
        string address, city, state, zip;
        ...
};
class Student
{
    private:
        StudentInfo personalData;
        ...
};
```

Member Initialization Lists

- Used in constructors for classes involved in aggregation.
- Allows constructor for enclosing class to pass arguments to the constructor of the enclosed class
- Notation:

```
owner_class(parameters) : owned_class(parameters) ;
```

Member Initialization Lists

Use:

```
class StudentInfo
{
    ...
};
class Student
{
    private:
        StudentInfo personalData;
    public:
        Student(string fname, lname) :
            StudentInfo(fname, lname);
};
```

Member Initialization Lists

- Member Initialization lists can be used to simplify the coding of constructors
- Should keep the entries in the initialization list in the same order as they are declared in the class

Aggregation Through Pointers

- A 'has-a' relationship can be implemented by owning a pointer to an object
- Can be used when multiple objects of a class may 'have' the same attribute for a member
 - ex: students who may have the same city/state/ zipcode
- Using pointers minimizes data duplication and saves space

Aggregation, Composition, and Object Lifetimes

- Aggregation represents the owner/owned relationship between objects.
- Composition is a form of aggregation in which the lifetime of the owned object is the same as that of the owner object
- Owned object is usually created as part of the owning object's constructor, destroyed as part of owning object's destructor

11.10 Inheritance

- **Inheritance** is a way of creating a new class by starting with an existing class and adding new members
- The new class can replace or extend the functionality of the existing class
- Inheritance models the 'is-a' relationship between classes

Inheritance - Terminology

- The existing class is called the **base class**
 - Alternates: **parent class**, **superclass**
- The new class is called the **derived class**
 - Alternates: **child class**, **subclass**

Inheritance Syntax and Notation

```
// Existing class
```

```
class Base
```

```
{
```

```
};
```

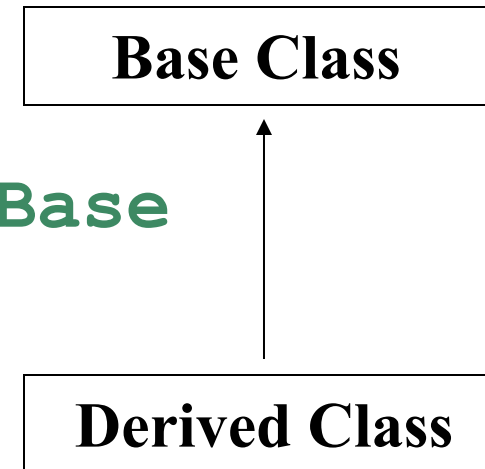
```
// Derived class
```

```
class Derived : public Base
```

```
{
```

```
};
```

Inheritance Class Diagram



Inheritance of Members

```
class Parent
{
    int a;
    void bf();
};
```

Objects of Parent have members

```
int a; void bf();
```

```
class Child : public
            Parent
{
    int c;
    void df();
};
```

Objects of Child have members

```
int a; void bf();
int c; void df();
```

11.11 Protected Members and Class Access

- **protected member access specification**: A class member labeled **protected** is accessible to member functions of derived classes as well as to member functions of the same class
- Like **private**, except accessible to members functions of derived classes

Base Class Access Specification

Base class access specification determines how `private`, `protected`, and `public` members of base class can be accessed by derived classes

Base Class Access

C++ supports three inheritance modes, also called base class access modes:

- public inheritance

```
class Child : public Parent { };
```

- protected inheritance

```
class Child : protected Parent{ };
```

- private inheritance

```
class Child : private Parent{ };
```

Base Class Access vs. Member Access Specification

Base class access is not the same as member access specification:

- Base class access: determine access for inherited members
- Member access specification: determine access for members defined in the class

Member Access Specification

Specified using the keywords

private, protected, public

```
class MyClass
{
    private: int a;
    protected: int b; void fun();
    public: void fun2();
};
```

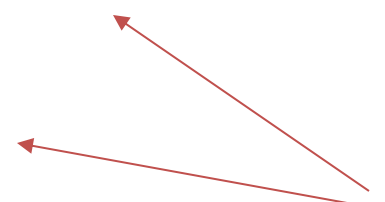
Base Class Access Specification

```
class Child : public Parent
{
    protected:
        int a;
    public:
        Child();
};
```

base access



member access



Base Class Access Specifiers

- 1) **public** – object of derived class can be treated as object of base class (not vice-versa)
- 2) **protected** – more restrictive than **public**, but allows derived classes to know some of the details of parents
- 3) **private** – prevents objects of derived class from being treated as objects of base class.

Effect of Base Access

Base class members

How base class members
appear in derived class

```
private: x  
protected: y  
public: z
```

private
base class

```
x inaccessible  
private: y  
private: z
```

```
private: x  
protected: y  
public: z
```

protected
base class

```
x inaccessible  
protected: y  
protected: z
```

```
private: x  
protected: y  
public: z
```

public
base class

```
x inaccessible  
protected: y  
public: z
```

11.12 Constructors, Destructors and Inheritance

- By inheriting every member of the base class, a derived class object contains a base class object
- The derived class constructor can specify which base class constructor should be used to initialize the base class object

Order of Execution

- When an object of a derived class is created, the base class's constructor is executed first, followed by the derived class's constructor
- When an object of a derived class is destroyed, its destructor is called first, then that of the base class

Order of Execution

```
// Student - base class
// UnderGrad - derived class
// Both have constructors, destructors
int main()
{
    UnderGrad u1;
    ...
    return 0;
} // end main
```

Execute **Student**
constructor, then
execute
UnderGrad
constructor

Execute **UnderGrad**
destructor, then
execute **Student**
destructor

Passing Arguments to Base Class Constructor

- Allows selection between multiple base class constructors
- Specify arguments to base constructor on derived constructor heading
- Can also be done with inline constructors
- Must be done if base class has no default constructor

Passing Arguments to Base Class Constructor

```
class Parent {  
    int x, y;  
    public: Parent(int,int) ;  
};  
class Child : public Parent {  
    int z  
    public:  
    Child(int a) : Parent(a,a*a)  
    {z = a;}  
};
```

11.13 Overriding Base Class Functions

- **Overriding**: function in a derived class that has the *same name and parameter list* as a function in the base class
- Typically used to replace a function in base class with different actions in derived class
- Not the same as overloading – with overloading, the parameter lists must be different

Access to Overridden Function

- When a function is overridden, all objects of derived class use the overriding function.
- If necessary to access the overridden version of the function, it can be done using the scope resolution operator with the name of the base class and the name of the function:

`Student::getName();`

Thanks,..

See you next week isA,..